Chương 1 (Phần 1): Giới thiệu về an toàn phần mềm

1.1 Khái niệm an toàn phần mềm – 1.2 Vấn đề an toàn phần mềm – 1.3 Kiểm thử & kiểm chứng (I)

ThS. Vũ Quang Dũng

Khoa Hệ thống thông tin – Trường Đại học Phenikaa

CSE703093 – An toàn phần mềm

Nội dung Phần 1

1. 1.1 Các khái niệm về an toàn phần mềm
2. 1.2 Một số vấn đề về an toàn phần mềm
3. 1.3 Kiểm thử và kiểm chứng phần mềm (Phần I)

Vì sao an toàn phần mềm quan trọng?

- Phần mềm điều khiển hầu hết hạ tầng hiện đại: ngân hàng, y tế, giao thông, IoT, ô tô, vũ khí...
- Một lỗi phần mềm nhỏ $\Rightarrow$ hậu quả lớn: rò rỉ dữ liệu, gián đoạn dịch vụ, thiệt hại tài chính, thậm chí nguy hiểm tính mạng
- Ví dụ nổi tiếng: lỗi tràn bộ nhớ đệm (buffer overflow) trong OpenSSL dẫn tới lỗ hổng Heartbleed (2014), cho phép đọc trộm bộ nhớ tiến trình
- Phần lớn lỗ hổng bảo mật **bắt nguồn từ lỗi lập trình**, không phải từ thuật toán mật mã yếu

Câu hỏi

Theo bạn, đâu là nguyên nhân phổ biến nhất gây ra lỗ hổng bảo mật trong phần mềm?

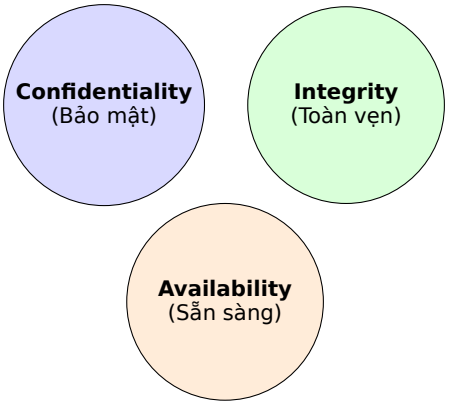
An toàn (Security) vs. An toàn vận hành (Safety) vs. Độ tin cậy (Reliability)

Thuật ngữ	**Ý nghĩa**
Security	Bảo vệ hệ thống trước các *tác nhân có chủ đích* (attacker)
Safety	Đảm bảo hệ thống không gây hại ngay cả khi *không có* tác nhân tấn công (lỗi ngẫu nhiên)
Reliability	Hệ thống hoạt động đúng chức năng liên tục trong điều kiện xác định

Ví dụ

Một hệ thống điều khiển máy bay có thể *an toàn vận hành* (không rơi do lỗi ngẫu nhiên) nhưng vẫn *không an toàn bảo mật* nếu tin tặc chiếm quyền điều khiển từ xa.

Mô hình CIA – Ba thuộc tính cốt lõi



Confidentiality – Bảo mật

Định nghĩa

Confidentiality: đảm bảo thông tin chỉ được tiết lộ cho các thực thể được ủy quyền.

- Vi phạm: rò rỉ dữ liệu người dùng, đọc trộm bộ nhớ tiến trình (ví dụ Heartbleed)
- Cơ chế bảo vệ: mã hóa (encryption), kiểm soát truy cập (access control), che dữ liệu nhạy cảm (data masking)

Integrity – Toàn vẹn

Định nghĩa

Integrity: đảm bảo dữ liệu/chương trình không bị thay đổi trái phép hoặc ngoài ý muốn.

- Vi phạm: sửa đổi giá trị biến qua lỗi tràn bộ nhớ, chèn mã độc, tấn công *man-in-the-middle*
- Cơ chế bảo vệ: hàm băm (hash), chữ ký số (digital signature), kiểm tra ràng buộc dữ liệu (input validation)

Availability – Sẵn sàng

Định nghĩa

Availability: đảm bảo hệ thống/dữ liệu luôn sẵn sàng phục vụ người dùng hợp lệ khi cần.

- Vi phạm: tấn công từ chối dịch vụ (Denial of Service – DoS/DDoS), lỗi vòng lặp vô hạn do lỗi lập trình
- Cơ chế bảo vệ: dự phòng (redundancy), cân bằng tải, giới hạn tài nguyên (rate limiting)

Mở rộng mô hình CIA

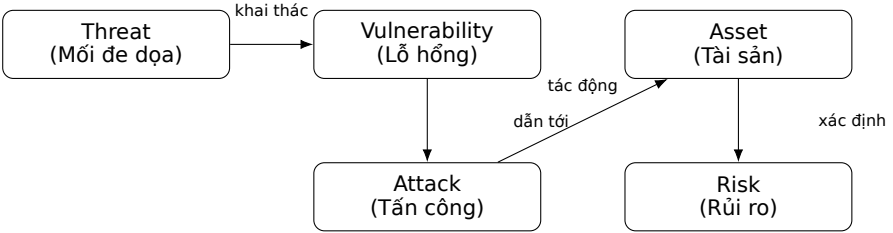
Ngoài CIA, các thuộc tính bảo mật khác thường được xét:

- **Authenticity** (Xác thực): xác nhận danh tính đúng của chủ thể/dữ liệu
- **Non-repudiation** (Không thể chối bỏ): chủ thể không thể phủ nhận hành động đã thực hiện
- **Accountability** (Trách nhiệm giải trình): mọi hành động có thể truy vết đến chủ thể thực hiện

Các khái niệm nền tảng: Asset – Threat – Vulnerability – Risk

Khái niệm	Định nghĩa
Asset (Tài sản)	Bất kỳ thứ gì có giá trị cần bảo vệ: dữ liệu, mã nguồn, uy tín, dịch vụ
Threat (Mối đe dọa)	Khả năng xảy ra sự kiện gây hại cho tài sản
Vulnerability (Lỗ hổng)	Điểm yếu trong hệ thống có thể bị khai thác
Attack (Tấn công)	Hành động cố ý khai thác lỗ hổng
Exploit	Đoạn mã/kỹ thuật cụ thể dùng để khai thác lỗ hổng
Risk (Rủi ro)	Xác suất và mức độ thiệt hại nếu mối đe dọa xảy ra

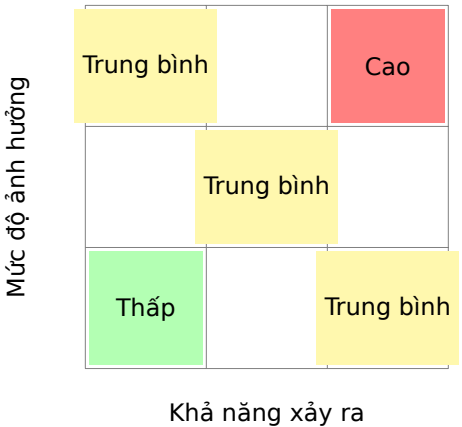
Quan hệ giữa các khái niệm



Ví dụ

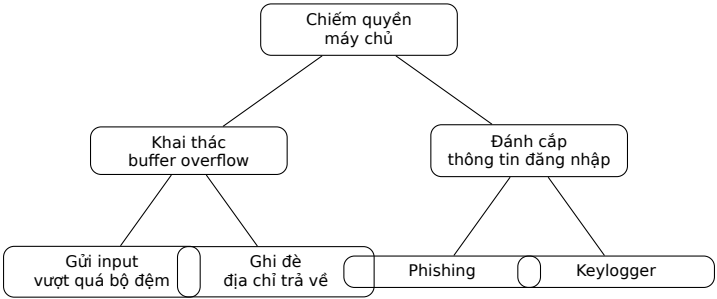
Buffer overflow (*vulnerability*) trong hàm xử lý input có thể bị kẻ tấn công (*threat agent*) khai thác qua *exploit* cụ thể để chiếm quyền điều khiển chương trình (*attack*), gây rủi ro mất dữ liệu (*risk*) trên *asset* là máy chủ.

Rủi ro = Khả năng xảy ra × Mức độ ảnh hưởng



Ma trận rủi ro giúp ưu tiên xử lý: lỗ hổng có khả năng khai thác cao *và* ảnh hưởng nghiêm trọng cần được vá trước tiên.

Cây tấn công (Attack Tree) – ví dụ



Attack tree phân rã mục tiêu tấn công thành các bước con, hỗ trợ phân tích rủi ro có hệ thống.

Authentication vs. Authorization

Authentication	Xác minh *bạn là ai* (mật khẩu, sinh trắc học, chứng chỉ số)
Authorization	Xác định *bạn được phép làm gì* sau khi đã xác thực (quyền truy cập)

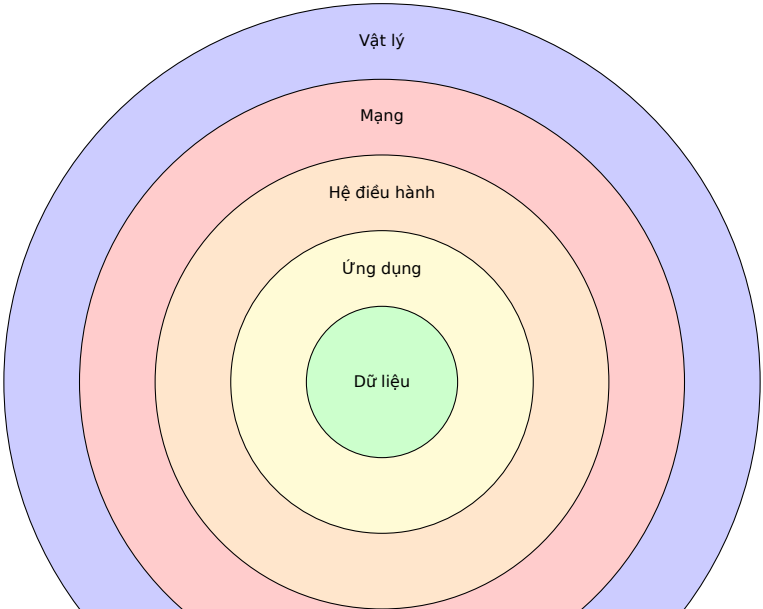
Ví dụ

Đăng nhập vào hệ thống bằng tài khoản (authentication) không có nghĩa bạn được phép xóa dữ liệu của người khác (authorization) – đây là hai bước độc lập, thường bị nhầm lẫn khi thiết kế phần mềm.

Nguyên tắc đặc quyền tối thiểu (Principle of Least Privilege)

- Mỗi thành phần (người dùng, tiến trình, module) chỉ nên có **quyền tối thiểu** cần thiết để hoàn thành công việc
- Giảm thiểu thiệt hại nếu thành phần đó bị xâm nhập
- Ví dụ vi phạm: chạy web server với quyền `root`; một lỗ hổng nhỏ trong ứng dụng web có thể dẫn đến chiếm toàn bộ hệ thống

Phòng thủ theo chiều sâu (Defense in Depth)



Security Policy và Security Model

- **Security Policy**: tập quy tắc mô tả *điều gì* được phép/không được phép trong hệ thống (mức tổ chức)
- **Security Model**: biểu diễn hình thức (formal) của chính sách bảo mật, dùng để *chứng minh* hệ thống tuân thủ chính sách
- Ví dụ mô hình: Bell-LaPadula (bảo mật đa cấp), Biba (toàn vẹn), mô hình dựa trên **Kripke structure** (sẽ học ở phần 1.5)

Trust và Trust Boundary

Định nghĩa

Trust boundary: ranh giới mà dữ liệu/lệnh đi qua đó cần được xác thực/kiểm tra lại vì mức độ tin cậy thay đổi.

- Dữ liệu từ người dùng (untrusted) đi vào logic xử lý (trusted) $\Rightarrow$ luôn phải **kiểm tra đầu vào** tại ranh giới này
- Phần lớn lỗ hổng bảo mật phần mềm xuất hiện do **thiếu kiểm tra** tại trust boundary

Mô hình hóa mối đe dọa: STRIDE

Chữ cái	Mối đe dọa	Vi phạm thuộc tính
S	Spoofing	Authenticity
T	Tampering	Integrity
R	Repudiation	Non-repudiation
I	Information Disclosure	Confidentiality
D	Denial of Service	Availability
E	Elevation of Privilege	Authorization

STRIDE (Microsoft) là khung phân loại mối đe dọa phổ biến khi phân tích thiết kế phần mềm.

Security by Design vs. Security as Afterthought

- **Afterthought**: bảo mật được "vá" sau khi phần mềm đã hoàn thiện $\rightarrow$ chi phí sửa lỗi cao, thường không triệt để
- **By Design**: bảo mật được tích hợp từ giai đoạn thiết kế, mô hình hóa mối đe dọa, kiểm chứng hình thức
- Học phần này tập trung vào các kỹ thuật giúp phát hiện lỗi **sớm**: kiểm chứng, kiểm thử, phân tích mã nguồn

Ví dụ định lượng: Tính Risk

Giả sử một lỗ hổng SQL Injection trên hệ thống thanh toán:

- Khả năng bị khai thác (Likelihood): Cao (input dễ thao túng, công cụ tấn công có sẵn) $\rightarrow$ điểm 9/10
- Mức độ ảnh hưởng (Impact): Nghiêm trọng (lộ toàn bộ dữ liệu thẻ) $\rightarrow$ điểm 10/10
- Risk score $=$ Likelihood $\times$ Impact $= 9 \times 10 = 90/100 \Rightarrow$ mức **Cao**, cần vá ngay lập tức

Bài tập: Điền ma trận rủi ro

Cho 3 lỗ hổng sau, hãy xác định vị trí trên ma trận rủi ro (Thấp/Trung bình/Cao) đã học:

1. Lỗi định dạng ngày tháng gây crash ứng dụng nội bộ, ít người dùng
2. Lỗ hổng cho phép đọc file cấu hình chứa mật khẩu, dễ khai thác từ xa
3. Lỗi hiếm gặp, chỉ xảy ra khi 2 luồng chạy đồng thời với xác suất rất thấp, nhưng có thể xóa dữ liệu

Đáp án bài tập ma trận rủi ro

1. Likelihood thấp, Impact thấp $\Rightarrow$ **Thấp**
2. Likelihood cao, Impact cao $\Rightarrow$ **Cao**
3. Likelihood thấp, Impact cao $\Rightarrow$ **Trung bình** (vẫn cần xử lý do hậu quả nghiêm trọng dù hiếm gặp)

Mô hình bảo mật hình thức: Bell–LaPadula

Định nghĩa

Mô hình **Bell–LaPadula** tập trung vào **Confidentiality**, dựa trên các mức phân loại (classification levels).

- *Simple Security Property* (“no read up”): chủ thể mức thấp không được đọc dữ liệu mức cao hơn
- **Property* (“no write down”): chủ thể mức cao không được ghi dữ liệu xuống mức thấp hơn
- Ứng dụng: hệ thống quân sự/chính phủ phân loại tài liệu Mật, Tối mật...

Mô hình bảo mật hình thức: Biba

Định nghĩa

Mô hình **Biba** tập trung vào **Integrity**, đối lập với Bell–LaPadula.

- *Simple Integrity Property* (“no read down”): không đọc dữ liệu có độ tin cậy thấp hơn
- **-Integrity Property* (“no write up”): không ghi lên dữ liệu có độ tin cậy cao hơn
- Ứng dụng: bảo vệ tiến trình hệ điều hành khỏi bị dữ liệu người dùng (kém tin cậy) làm hỏng

Mô hình kiểm soát truy cập: DAC, MAC, RBAC

DAC	*Discretionary Access Control*: chủ sở hữu tài nguyên tự quyết định ai được truy cập (vd: quyền file Unix `rwx`)
MAC	*Mandatory Access Control*: hệ thống áp đặt chính sách truy cập tập trung, người dùng không thể thay đổi (vd: SELinux)
RBAC	*Role-Based Access Control*: quyền được gán theo *vai trò*, người dùng nhận quyền qua vai trò được gán

Ví dụ RBAC trong thiết kế phần mềm

```
roles = {
    "admin": {"create", "read", "update", "delete"},
    "editor": {"create", "read", "update"},
    "viewer": {"read"}
}

def check_permission(user_role, action):
    return action in roles.get(user_role, set())
```

Thiết kế RBAC rõ ràng giúp tránh lỗi *broken access control* – một trong những lỗ hổng phổ biến nhất theo OWASP.

Bài tập: Phân loại STRIDE

Một ứng dụng cho phép người dùng đổi email nhận thông báo mà không cần xác thực lại mật khẩu. Kẻ tấn công đã đăng nhập có thể đổi email của tài khoản khác qua lỗi IDOR (Insecure Direct Object Reference).
Hỏi: Đây là loại mối đe dọa nào trong STRIDE?

Đáp án: Phân loại STRIDE

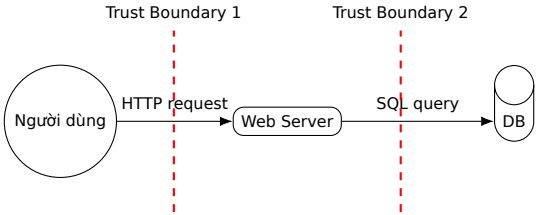
- Đây chủ yếu là **Elevation of Privilege** (truy cập/sửa dữ liệu ngoài phạm vi được phép)
- Đồng thời cũng có yếu tố **Tampering** (dữ liệu tài khoản người khác bị thay đổi trái phép)
- Nguyên nhân gốc: thiếu kiểm tra *authorization* ở tầng server (chỉ dựa vào ID gửi từ client)

Quy trình mô hình hóa mối đe dọa (Threat Modeling)

Bốn câu hỏi cốt lõi (theo phương pháp luận phổ biến):

1. Chúng ta đang xây dựng cái gì? (mô hình hóa hệ thống, data flow diagram)
2. Điều gì có thể sai? (liệt kê mối đe dọa, dùng STRIDE)
3. Chúng ta sẽ làm gì với điều đó? (biện pháp giảm thiểu)
4. Chúng ta đã làm tốt việc phân tích chưa? (rà soát lại)

Ví dụ Data Flow Diagram với Trust Boundary



Mỗi lần dữ liệu đi qua ranh giới tin cậy, cần xác thực/kiểm tra lại đầu vào.

Bài tập: Xác định Trust Boundary

Cho hệ thống 3 lớp: *Trình duyệt* $\rightarrow$ *API Gateway* $\rightarrow$ *Microservice nội bộ* $\rightarrow$ *Cơ sở dữ liệu*.
Hỏi: Có bao nhiêu trust boundary hợp lý? Tại lớp nào cần kiểm tra input nghiêm ngặt nhất?

Đáp án: Xác định Trust Boundary

- 3 ranh giới: Trình duyệt→API Gateway; API Gateway→Microservice; Microservice→DB
- Nghiêm ngặt nhất: **Trình duyệt → API Gateway** – vì đây là nơi dữ liệu *chưa được tin cậy* (untrusted, từ người dùng bất kỳ) lần đầu đi vào hệ thống
- Lưu ý: dù các lớp sau "nội bộ" hơn, thực hành tốt vẫn khuyến nghị kiểm tra input ở *mọi* ranh giới (defense in depth)

Phân loại tài sản (Asset Classification)

- **Dữ liệu**: thông tin cá nhân, tài chính, mã nguồn, khóa mật mã
- **Dịch vụ**: tính khả dụng của hệ thống, uy tín thương hiệu
- **Hạ tầng**: máy chủ, thiết bị mạng, thiết bị IoT
- Việc phân loại tài sản theo mức độ nhạy cảm giúp phân bổ nguồn lực bảo mật hợp lý (asset có giá trị cao cần nhiều lớp bảo vệ hơn)

Bài tập tổng hợp 1.1

Một trường đại học triển khai hệ thống điểm danh bằng khuôn mặt (face recognition).

1. Liệt kê 2 tài sản (asset) cần bảo vệ
2. Liệt kê 2 mối đe dọa (threat) tiềm ẩn
3. Đề xuất 1 biện pháp theo nguyên tắc least privilege

Gợi ý đáp án bài tập tổng hợp 1.1

- Asset: dữ liệu sinh trắc học khuôn mặt, dữ liệu điểm danh
- Threat: giả mạo khuôn mặt (spoofing) để điểm danh hộ, rò rỉ dữ liệu sinh trắc học
- Least privilege: chỉ nhân viên phòng đào tạo được truy cập dữ liệu điểm danh chi tiết, giảng viên chỉ xem được lớp mình dạy

Tổng kết 1.1

- An toàn phần mềm khác an toàn vận hành và độ tin cậy
- Mô hình CIA (+ Authenticity, Non-repudiation) là nền tảng của mọi phân tích bảo mật
- Asset – Threat – Vulnerability – Attack – Risk là chuỗi khái niệm cốt lõi
- Các nguyên tắc thiết kế: least privilege, defense in depth, trust boundary

Câu hỏi

Hãy phân loại một cuộc tấn công DDoS vào 3 thuộc tính CIA và giải thích.

Vì sao phần mềm lỗi là nguyên nhân bảo mật chính?

- Đa số các vụ tấn công thành công khai thác **lỗi lập trình**, không phải lỗi thuật toán mật mã
- Phần mềm ngày càng phức tạp $\Rightarrow$ bề mặt tấn công (attack surface) lớn hơn
- Áp lực thời gian phát triển thường khiến kiểm tra bảo mật bị xem nhẹ

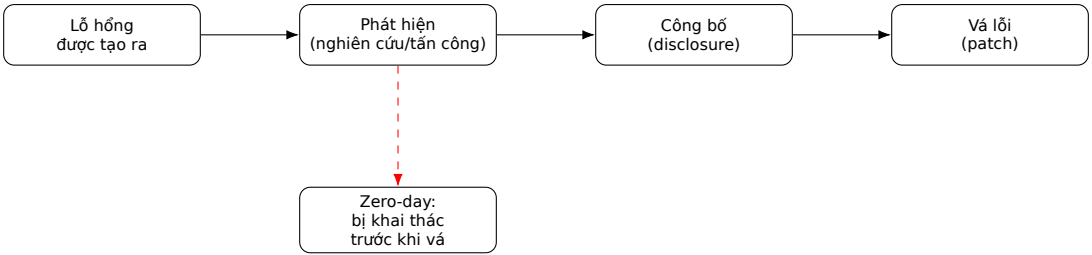
CWE và CVE

CWE	*Common Weakness Enumeration*: danh mục chuẩn hóa các **loại điểm yếu** phần mềm (ví dụ CWE-120: Buffer Overflow)
CVE	*Common Vulnerabilities and Exposures*: danh mục các **lỗ hổng cụ thể** đã được công bố trong sản phẩm phần mềm thực tế

Ví dụ

CVE-2014-0160 (Heartbleed) là một lỗ hổng cụ thể thuộc loại CWE-125 (Out-of-bounds Read).

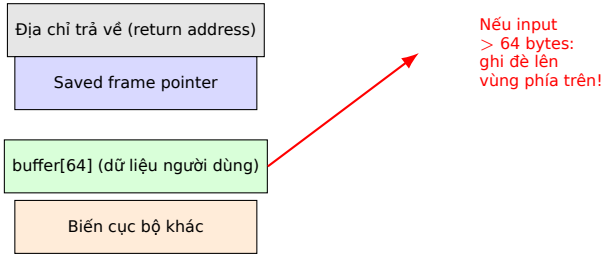
Vòng đời lỗ hổng bảo mật



Phân loại lỗ hổng phần mềm phổ biến

- Lỗi an toàn bộ nhớ (memory safety): buffer overflow, use-after-free, double free
- Lỗi kiểm tra đầu vào (input validation): injection (SQL, command), path traversal
- Lỗi số học: integer overflow/underflow
- Lỗi đồng bộ hóa: race condition, TOCTOU (Time-of-check to time-of-use)
- Lỗi xử lý ngoại lệ/lỗi: improper error handling, thông tin nhạy cảm bị lộ qua thông báo lỗi
- Lỗi logic nghiệp vụ (business logic flaws)

Buffer Overflow – Minh họa cấu trúc stack



Khi dữ liệu ghi vào `buffer` vượt quá kích thước cấp phát, các vùng nhớ liền kề (kể cả địa chỉ trả về) có thể bị ghi đè – đây là cơ chế cốt lõi của lỗi tràn bộ đệm.

Buffer Overflow – Ví dụ mã C dễ tổn thương

```c
void copy_input(char *user_input) {
    char buffer[64];
    strcpy(buffer, user_input);  // Khong kiem tra do dai!
    printf("Da nhan: %s\n", buffer);
}
```

Lưu ý

`strcpy` không kiểm tra kích thước đích. Nếu `user_input` dài hơn 64 byte, dữ liệu sẽ tràn ra ngoài `buffer`.

Buffer Overflow – Phiên bản an toàn hơn

```
void copy_input_safe(const char *user_input) {
    char buffer[64];
    strncpy(buffer, user_input, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';  // Dam bao ket thuc chuoi
    printf("Da nhan: %s\n", buffer);
}
```

Sử dụng `strncpy` kèm giới hạn độ dài và đảm bảo null-terminator giúp giảm rủi ro tràn bộ đệm (nội dung chi tiết ở Chương 2).

Injection – Ví dụ SQL Injection

```
# De bi tan cong:
query = "SELECT * FROM users WHERE name = '" + username + "'"
cursor.execute(query)

# Neu username = "' OR '1'='1"
# => SELECT * FROM users WHERE name = '' OR '1'='1'
# Tra ve TOAN BO du lieu nguoi dung!
```

Lưu ý

Nối chuỗi trực tiếp dữ liệu người dùng vào câu truy vấn cho phép kẻ tấn công thay đổi **cấu trúc logic** của câu lệnh SQL.

Injection – Cách phòng chống: Parameterized Query

```
# An toan hon: dung tham so hoa (prepared statement)
query = "SELECT * FROM users WHERE name = ?"
cursor.execute(query, (username,))
```

Cơ chế *parameterized query* tách biệt hoàn toàn dữ liệu và cấu trúc câu lệnh, ngăn chặn injection tại tầng driver cơ sở dữ liệu.

Integer Overflow

```
unsigned char len = 250;
len = len + 10;   // Ket qua: 4 (250 + 10 = 260, wrap mod 256)
char buf[len];    // Cap phat bo nho qua NHO so voi du dinh!
```

- Kiểu dữ liệu có kích thước cố định $\Rightarrow$ phép cộng có thể “quay vòng” (wrap-around)
- Hệ quả: cấp phát bộ nhớ sai kích thước, dẫn tới buffer overflow gián tiếp

Race Condition / TOCTOU

```c
if (access("file.txt", W_OK) == 0) {   // Time-of-check
    // ... mot tien trinh khac doi ten file thanh symlink toi /etc/passwd
    fd = open("file.txt", O_WRONLY);   // Time-of-use
    write(fd, data, len);
}
```

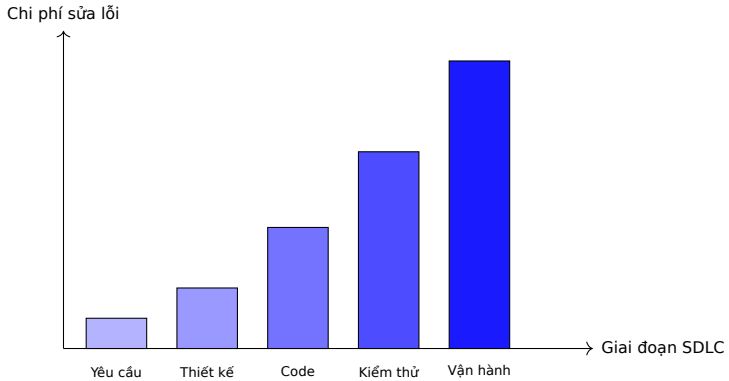
Lưu ý

Giữa bước *kiểm tra* (check) và bước *sử dụng* (use), trạng thái hệ thống có thể bị thay đổi bởi tiến trình khác – đây là lỗi **Time-of-check to time-of-use (TOCTOU)**.

Improper Error Handling

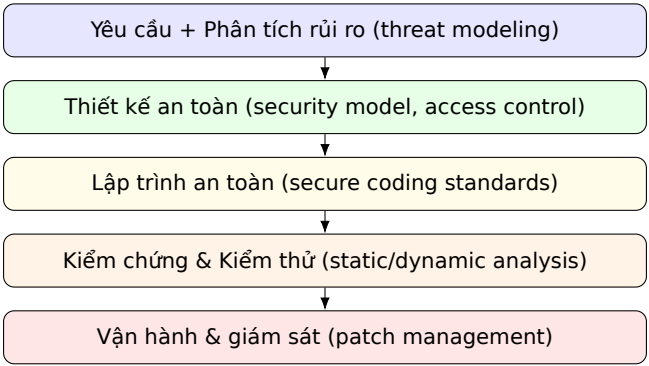
- Thông báo lỗi chi tiết (stack trace, đường dẫn hệ thống, câu truy vấn SQL) có thể lộ thông tin nhạy cảm cho kẻ tấn công
- Xử lý lỗi không đầy đủ (bỏ qua exception) có thể khiến hệ thống ở trạng thái không nhất quán, mất toàn vẹn dữ liệu
- Nguyên tắc: *fail securely* – khi có lỗi, hệ thống nên chuyển về trạng thái an toàn mặc định

Chi phí sửa lỗi theo giai đoạn phát triển



Phát hiện lỗi càng muộn trong vòng đời phát triển, chi phí khắc phục càng tăng theo cấp số nhân – đây là lý do kiểm chứng/kiểm thử sớm (Chương 1.3 trở đi) có giá trị lớn.

Secure Software Development Lifecycle (SDLC)



Nghiên cứu tình huống: Heartbleed (CVE-2014-0160)

- Lỗ hổng trong thư viện OpenSSL (chức năng TLS heartbeat), thuộc loại *out-of-bounds read* (CWE-125)
- Nguyên nhân gốc: không kiểm tra độ dài trường dữ liệu do client cung cấp trước khi copy bộ nhớ
- Hậu quả: kẻ tấn công có thể đọc tới 64KB bộ nhớ tiến trình máy chủ mỗi lần gửi yêu cầu, tiềm ẩn rò rỉ khóa riêng tư (private key)
- Bài học: một dòng kiểm tra độ dài thiếu sót có thể gây ảnh hưởng diện rộng toàn cầu

Ánh xạ tới OWASP – các nhóm lỗi liên quan

- Injection (SQL, OS command, LDAP...)
- Broken Access Control
- Cryptographic Failures
- Security Misconfiguration
- Vulnerable and Outdated Components

Nhiều mục trong danh sách OWASP Top 10 có gốc rễ chung với các lớp lỗi phần mềm đã trình bày (injection, input validation, logic lỗi).

Use-After-Free

```c
char *p = malloc(64);
free(p);
strcpy(p, "du lieu moi");   // Ghi vao bo nho DA GIAI PHONG!
```

Lưu ý

Sau `free(p)`, vùng nhớ có thể được cấp lại cho mục đích khác. Ghi/đọc qua con trỏ p lúc này có thể làm hỏng dữ liệu của đối tượng khác hoặc bị kẻ tấn công lợi dụng (CWE-416).

Double Free

```
char *p = malloc(64);
free(p);
free(p);   // Giai phong lan thu 2 -- lam hong bo quan ly heap!
```

Giải phóng bộ nhớ hai lần có thể phá vỡ cấu trúc dữ liệu nội bộ của bộ cấp phát heap, dẫn tới hành vi không xác định hoặc bị khai thác để kiểm soát bộ nhớ (CWE-415).

Path Traversal

```
filename = request.args.get("file")     # vd: "../../etc/passwd"
with open(f"/uploads/{filename}") as f:
    return f.read()
```

Lưu ý

Không kiểm tra/chuẩn hóa đường dẫn cho phép kẻ tấn công dùng `../` để thoát khỏi thư mục dự kiến và đọc file hệ thống bất kỳ (CWE-22).

Command Injection

```python
import os
filename = request.args.get("name")
os.system(f"cat {filename}")   # Neu name = "a.txt; rm -rf /"
```

Nối trực tiếp input người dùng vào lệnh hệ điều hành cho phép kẻ tấn công chèn thêm lệnh tùy ý (CWE-78) – tương tự SQL injection nhưng ở tầng hệ điều hành.

Case study: WannaCry (2017)

- Ransomware lan truyền qua lỗ hổng **EternalBlue** trong giao thức SMB của Windows
- Lỗ hổng thuộc loại lỗi bộ nhớ (buffer overflow) trong xử lý gói tin SMB
- Lan rộng toàn cầu trong vài giờ do khai thác tự động, không cần tương tác người dùng
- Bài học: một lỗ hổng bộ nhớ chưa vá trong dịch vụ mạng phổ biến có thể gây thiệt hại diện rộng

Case study: Log4Shell (CVE-2021-44228)

- Lỗ hổng trong thư viện ghi log Java phổ biến (Log4j)
- Cho phép thực thi mã từ xa (Remote Code Execution) chỉ bằng cách gửi một chuỗi log đặc biệt
- Nguyên nhân: tính năng tra cứu JNDI được xử lý ngay trong chuỗi log mà không kiểm soát nguồn gốc dữ liệu
- Ảnh hưởng hàng triệu hệ thống dùng thư viện này gián tiếp qua các framework khác

Bài tập: Xác định lớp lỗi

Đoạn mã sau thuộc lớp lỗi nào đã học?

```
int idx = get_user_index();     // Khong kiem tra pham vi
data[idx] = 42;
```

Đáp án bài tập

- Đây là lỗi **out-of-bounds write** (CWE-787) nếu `idx` không được kiểm tra nằm trong phạm vi mảng
- Thuộc nhóm rộng hơn: *memory safety* / *input validation*
- Cách sửa: kiểm tra `0 <= idx < len(data)` trước khi ghi

Giới thiệu CVSS (Common Vulnerability Scoring System)

- Hệ thống chấm điểm chuẩn hóa mức độ nghiêm trọng của lỗ hổng, thang điểm 0–10
- Các yếu tố chính: vector tấn công (network/local), độ phức tạp khai thác, quyền cần có, mức ảnh hưởng CIA
- CVE thường đi kèm điểm CVSS để tổ chức ưu tiên vá lỗi

Ví dụ tính điểm CVSS (đơn giản hóa)

Lỗ hổng khai thác từ xa (Network), không cần xác thực, ảnh hưởng cao tới Confidentiality và Integrity:

- Attack Vector: Network (điểm cao)
- Privileges Required: None (điểm cao)
- Impact C/I: High/High
- $\Rightarrow$ Điểm CVSS tổng hợp thường rơi vào khoảng **9.0–10.0 (Critical)**

Chuẩn lập trình an toàn (Secure Coding Standards)

- **CERT C/C++ Coding Standard**: quy tắc tránh các lỗi bộ nhớ, số học phổ biến trong C/C++
- **MISRA C**: chuẩn dùng nhiều trong hệ thống nhúng, ô tô, yêu cầu an toàn cao
- **OWASP Secure Coding Practices**: hướng dẫn chung cho ứng dụng web
- Việc tuân thủ chuẩn giúp giảm đáng kể số lượng lỗ hổng loại memory-safety và injection

Công cụ phân tích tĩnh (xem trước Chương 3)

Công cụ	Ngôn ngữ	Phát hiện
cppcheck	C/C++	Lỗi bộ nhớ, cú pháp đáng ngờ
clang-tidy	C/C++	Vi phạm coding standard
bandit	Python	Lỗ hổng bảo mật phổ biến

Các công cụ này sẽ được thực hành chi tiết trong dự án Python của Chương 3.

Tổng kết 1.2

- CWE phân loại *lớp lỗi*, CVE ghi nhận *lỗ hổng cụ thể*
- Các lớp lỗi phổ biến: memory safety, injection, integer overflow, race condition, error handling
- Chi phí sửa lỗi tăng mạnh theo giai đoạn phát triển $\Rightarrow$ cần phát hiện sớm

Câu hỏi

Đoạn mã `strcpy(buffer, input)` thuộc CWE nào? Hãy đề xuất cách sửa.

Testing vs. Verification vs. Validation

Verification	“Are we building the product right?” – Kiểm tra sản phẩm có tuân thủ đặc tả không
Validation	“Are we building the right product?” – Kiểm tra sản phẩm có đáp ứng nhu cầu người dùng không
Testing	Một kỹ thuật cụ thể để thực hiện verification/validation bằng cách *thực thi* chương trình

Định nghĩa Testing (IEEE)

Định nghĩa

Software Testing: quá trình thực thi chương trình với mục đích tìm ra lỗi (defects), nhằm đánh giá và cải thiện chất lượng phần mềm.

Mục tiêu của testing:

- Phát hiện lỗi (finding defects)
- Xây dựng niềm tin về mức chất lượng phần mềm
- Ngăn ngừa lỗi (test sớm giúp phát hiện vấn đề trong yêu cầu/thiết kế)
- Cung cấp thông tin để ra quyết định (release hay không)

Nguyên lý cơ bản của Testing

- Testing chỉ có thể chứng minh **sự tồn tại của lỗi**, không thể chứng minh **không có lỗi**
- Kiểm thử toàn bộ (exhaustive testing) thường bất khả thi với chương trình thực tế
- Testing cần được lập kế hoạch và thiết kế có hệ thống, không chỉ chạy ngẫu nhiên
- Lỗi thường tập trung (defect clustering): một số module chứa nhiều lỗi hơn phần còn lại

Static vs. Dynamic Techniques

Static (Tĩnh)
Phân tích mà *không* thực thi chương trình
(code review, static analysis, model checking)

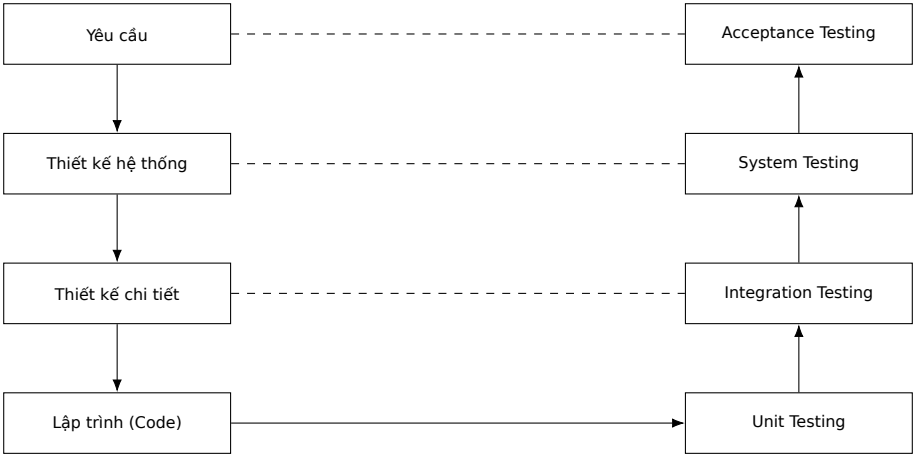
Dynamic (Động)
Phân tích bằng cách *thực thi* chương trình
(testing, fuzzing, monitoring)

Cả hai kỹ thuật đều cần thiết – static phát hiện lỗi sớm và bao phủ nhiều đường thực thi lý thuyết, dynamic phát hiện lỗi thực tế khi chạy với dữ liệu cụ thể.

Verification Techniques – Toàn cảnh

- **Formal proof (Chứng minh hình thức)**: chứng minh toán học chương trình thỏa mãn đặc tả
- **Model checking**: kiểm tra tự động một mô hình hữu hạn trạng thái có thỏa mãn thuộc tính hay không (chi tiết ở 1.5, Chương 3)
- **Static analysis**: phân tích mã nguồn/mã máy để phát hiện lỗi mà không cần chạy chương trình (Chương 3)
- **Testing**: thực thi chương trình với các bộ dữ liệu đầu vào (test case) (Chương 4)
- **Code review / Peer review**: con người đọc và đánh giá mã nguồn thủ công

Mô hình chữ V (V-Model) trong kiểm thử



Mỗi giai đoạn phát triển tương ứng với một cấp độ kiểm thử – kiểm thử được thiết kế song song với thiết kế, không chỉ sau khi code xong.

Các mức độ kiểm thử (Test Levels)

Unit Testing	Kiểm thử từng đơn vị nhỏ nhất (hàm, phương thức) độc lập
Integration Testing	Kiểm thử tương tác giữa các module đã tích hợp
System Testing	Kiểm thử toàn bộ hệ thống theo yêu cầu đặc tả
Acceptance Testing	Kiểm thử theo góc nhìn người dùng/khách hàng trước khi bàn giao

Black-box Testing vs. White-box Testing

	Black-box	**White-box**	
Cơ sở thiết kế test	Đặc tả chức năng	Cấu trúc mã nguồn nội bộ	Chi tiết kỹ thuật
Kiến thức cần có	Không cần biết code	Cần đọc hiểu code	
Kỹ thuật điển hình	Equivalence partitioning, boundary value	Statement/branch/path coverage	

white-box (code coverage, fuzzing) sẽ được trình bày sâu ở Chương 4.

Equivalence Partitioning – Ví dụ

Hàm kiểm tra tuổi hợp lệ để đăng ký: $0 \leq \text{age} \leq 120$.



Thay vì thử mọi giá trị tuổi, ta chỉ cần chọn một giá trị đại diện cho mỗi lớp tương đương – giảm số test case mà vẫn đảm bảo độ bao phủ hợp lý.

Boundary Value Analysis – Ví dụ

Với lớp hợp lệ $0 \leq \text{age} \leq 120$, các giá trị biên quan trọng nhất:

- -1 (ngay dưới biên dưới – kỳ vọng invalid)
- 0 (biên dưới – kỳ vọng valid)
- 120 (biên trên – kỳ vọng valid)
- 121 (ngay trên biên trên – kỳ vọng invalid)

Phần lớn lỗi lập trình (off-by-one) xảy ra chính tại các giá trị biên, nên đây là kỹ thuật hiệu quả để phát hiện lỗi với chi phí thấp.

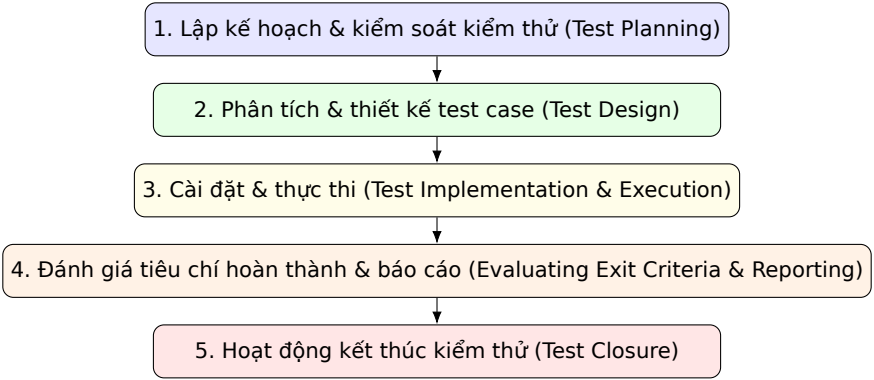
Soundness và Completeness (giới thiệu)

Sound (Đúng đắn)	Nếu kỹ thuật báo “không có lỗi”, thì thực sự không có lỗi (không có false negative bị bỏ sót về mặt lý thuyết)
Complete (Đầy đủ)	Nếu có lỗi thực sự tồn tại, kỹ thuật chắc chắn phát hiện ra được nó

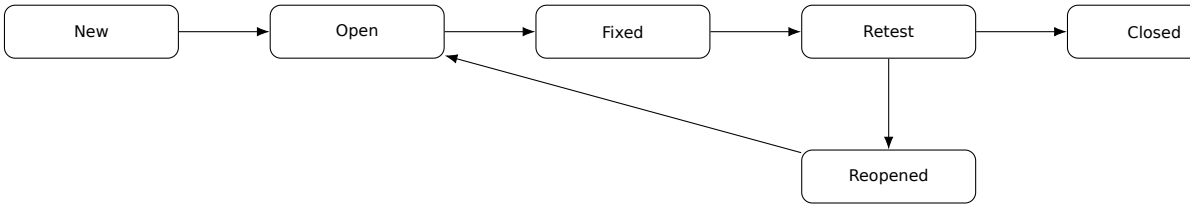
Lưu ý

Testing nói chung là *không đầy đủ* (không thể thử mọi trường hợp), trong khi model checking trên mô hình hữu hạn có thể đạt cả sound và complete đối với mô hình đó. Chi tiết sẽ học ở Chương 3.

Quy trình kiểm thử (Test Process)



Vòng đời lỗi (Defect Lifecycle)



Giới hạn của Testing đối với bảo mật

- Testing truyền thống tập trung vào *chức năng đúng theo đặc tả* – nhưng lỗ hổng bảo mật thường nằm ở những **hành vi ngoài đặc tả** (input không mong đợi, đường thực thi hiếm gặp)
- Kẻ tấn công chủ động tìm input "xấu" – khác với tư duy test case thông thường nhắm vào input "điển hình"
- Vì vậy cần bổ sung: phân tích tĩnh, model checking, fuzzing (kiểm thử với input được sinh tự động, ngẫu nhiên hoặc dẫn hướng)

Thành phần của một Test Case

ID	Định danh duy nhất cho test case
Precondition	Điều kiện cần có trước khi thực thi
Input	Dữ liệu đầu vào cụ thể
Expected Output	Kết quả mong đợi
Postcondition	Trạng thái hệ thống sau khi thực thi

Ví dụ Test Case: Chức năng đăng nhập

ID	TC_LOGIN_01
Precondition	Tài khoản `user1/pass123` đã tồn tại
Input	`username=user1`, `password=pass123`
Expected Output	Đăng nhập thành công, chuyển tới trang chủ
Postcondition	Phiên làm việc (session) được tạo

Bài toán tổ hợp: Vì sao Exhaustive Testing bất khả thi

Một hàm nhận 2 số nguyên 32-bit làm đầu vào:

- Số tổ hợp đầu vào: $2^{32} \times 2^{32} = 2^{64} \approx 1.8 \times 10^{19}$
- Nếu chạy 1 tỷ test/giây, vẫn cần khoảng **584 năm** để thử hết
- $\Rightarrow$ Cần chọn lọc test case thông minh (equivalence partitioning, boundary value) thay vì thử toàn bộ

Formal Proof – Ví dụ minh họa (Hoare Logic, xem trước)

Bộ ba Hoare: $\{P\}\ S\ \{Q\}$ – nếu điều kiện trước P đúng, sau khi thực thi S thì điều kiện sau Q đúng.

$$\{x = a \wedge y = b\} \quad \texttt{tmp := x; x := y; y := tmp} \quad \{x = b \wedge y = a\}$$

Formal proof chứng minh chương trình *luôn* thỏa mãn đặc tả cho *mọi* đầu vào – khác với testing chỉ kiểm tra một số trường hợp cụ thể.

Bài tập: Static hay Dynamic?

Phân loại các kỹ thuật sau là Static hay Dynamic:

1. Đọc code thủ công tìm lỗi logic (code review)
2. Chạy chương trình với input ngẫu nhiên (fuzzing)
3. Dùng công cụ phân tích luồng dữ liệu mà không chạy chương trình
4. Đo code coverage khi chạy bộ test

Đáp án: Static hay Dynamic?

1. Code review → **Static**
2. Fuzzing → **Dynamic**
3. Phân tích luồng dữ liệu tĩnh → **Static**
4. Đo code coverage → **Dynamic** (cần chạy chương trình)

Định nghĩa

Regression Testing: kiểm thử lại phần mềm sau khi có thay đổi (sửa lỗi, thêm tính năng) để đảm bảo chức năng cũ không bị ảnh hưởng.

- Đặc biệt quan trọng với bảo mật: một bản vá có thể vô tình mở lại lỗ hổng cũ hoặc tạo lỗ hổng mới
- Thường được tự động hóa để chạy mỗi khi mã nguồn thay đổi (CI/CD)

Test Automation – Ví dụ với pytest

```python
def is_valid_age(age):
    return 0 <= age <= 120

def test_age_lower_boundary():
    assert is_valid_age(0) == True
    assert is_valid_age(-1) == False

def test_age_upper_boundary():
    assert is_valid_age(120) == True
    assert is_valid_age(121) == False
```

Test tự động hóa giúp chạy lại nhanh chóng mỗi khi mã nguồn thay đổi – nền tảng cho pipeline CI/CD an toàn.

Bài tập: Equivalence Partitioning cho mật khẩu

Quy tắc: mật khẩu hợp lệ có độ dài từ 8 đến 20 ký tự.
Hãy xác định các lớp tương đương (equivalence classes) cho độ dài mật khẩu.

Đáp án: Equivalence Partitioning cho mật khẩu

- Lớp không hợp lệ 1: độ dài < 8
- Lớp hợp lệ: $8 \leq$ độ dài ≤ 20
- Lớp không hợp lệ 2: độ dài > 20

Bài tập: Boundary Value Analysis cho mật khẩu

Với cùng quy tắc (8–20 ký tự), hãy liệt kê các giá trị biên cần kiểm thử.

Đáp án: Boundary Value Analysis cho mật khẩu

Các giá trị biên: 7 (invalid), 8 (valid), 20 (valid), 21 (invalid).
Đây chính là 4 test case tối thiểu để bao phủ các điểm dễ xảy ra lỗi off-by-one nhất.

Chuyển tiếp sang Phần 2

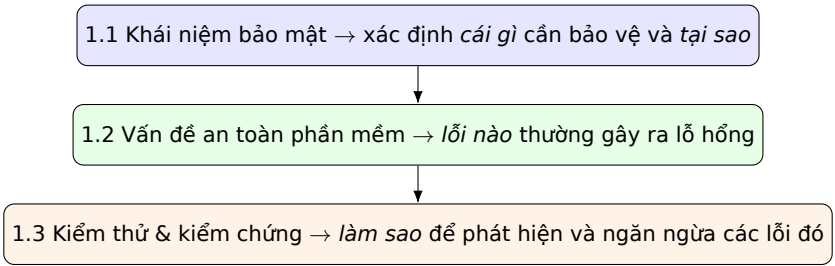
Phần 2 của Chương 1 sẽ trình bày:

- 1.4 Kiểm thử và kiểm chứng phần mềm (Phần II) – kỹ thuật testing nâng cao
- 1.5 Cấu trúc Kripke (Kripke Structure) – nền tảng hình thức cho model checking
- 1.6 Tổng kết Chương 1 và bài tập

Tổng kết 1.3 (Phần I)

- Testing $\neq$ Verification $\neq$ Validation
- Static vs Dynamic là hai trục kỹ thuật chính xuyên suốt học phần
- V-model gắn kết mức kiểm thử với giai đoạn phát triển
- Black-box (equivalence partitioning, boundary value) vs White-box (coverage)
- Testing về bản chất không đầy đủ – cần kết hợp với các kỹ thuật kiểm chứng hình thức

Liên hệ giữa 1.1, 1.2, 1.3



Tự đánh giá nhanh (Self-check)

Hãy tự trả lời (không cần nộp):

1. Phân biệt Vulnerability và Exploit bằng ví dụ của riêng bạn
2. Vì sao testing không thể chứng minh phần mềm không có lỗi?
3. Kể tên 2 lớp lỗi (CWE) đã học và cách phòng tránh tương ứng

Chuẩn bị dự án Python (module ch1_models)

Buổi thực hành sẽ bắt đầu xây dựng module đầu tiên của dự án xuyên suốt môn học:

- Cài đặt cấu trúc project `softsec-toolkit` (xem deck Python Project)
- Viết hàm mô phỏng *equivalence partitioning* và *boundary value analysis* tự động sinh test case từ khoảng giá trị hợp lệ
- Chuẩn bị nền tảng cho `ch1_models`: biểu diễn trạng thái hệ thống bảo mật (tiền đề cho Kripke Structure ở Phần 2)

Tài liệu tham khảo thêm cho Phần 1

- ISTQB Foundation Level Syllabus – các định nghĩa testing/verification chuẩn quốc tế
- MITRE CWE/CVE database (cwe.mitre.org, cve.mitre.org)
- OWASP Top 10 (owasp.org)
- Sách giáo trình chính: Graham et al., *Foundations of Software Testing*

Bài tập thực hành (chuẩn bị cho buổi sau)

1. Với hàm `copy_input` (slide Buffer Overflow), viết 3 test case theo boundary value analysis cho độ dài input
2. Vẽ lại attack tree cho kịch bản: “Đánh cắp dữ liệu khách hàng từ ứng dụng web bán hàng”
3. Phân loại 3 lỗ hổng CVE bất kỳ (tự tra cứu) theo CWE tương ứng

Hẹn gặp lại ở Phần 2

Chương 1 – Phần 2:
1.4 Testing & Verification (II) · 1.5 Kripke Structure · 1.6 Tổng kết